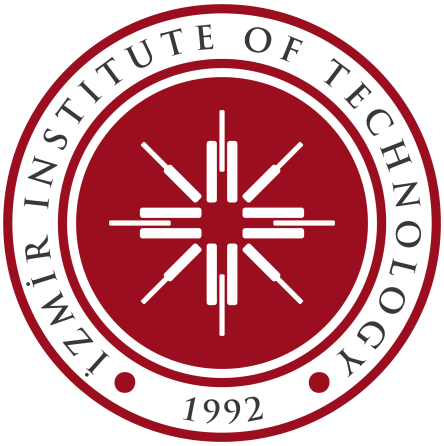
CENG318 Human Computer Interaction
Project Proposal
Spring 2025

Project Title: Kişisel: A Widget-Based Adaptive News Aggregation and Social Curation Platform

March 25, 2026



Group Members

- Sübhan Akbenli 300201129
- Berkay Fehmi Tekin 300201090
- Zübeyir Almaho 300201023
- Yasin Sezgin 300201007
- Ömer Faruk Teker 300201077

Abstract

This project proposes **Kişisel**, a widget-based, adaptive news aggregation and social curation platform designed to combat information overload, algorithmic filter bubbles, and the lack of user agency in modern digital reading. Shifting the paradigm from passive consumption to active "gatewatching," Kişisel allows users to build shareable "personal newspapers" using customizable spatial grids and AI-driven progressive summarization. Crucially, the system introduces rich-text Editorial Widgets, empowering users to embed personal commentary directly into their feeds. This proposal outlines the platform's theoretical HCI foundations, a four-stage user-centered design methodology, its full-stack architecture, and a mixed-methods usability evaluation plan designed to validate its impact on reading depth and user agency.

0 Introduction

Mainstream news aggregators typically rely on opaque algorithms that reduce users to passive consumers. This paradigm creates three critical human-computer interaction (HCI) challenges: cognitive fatigue from information overload, ideological isolation driven by filter bubbles **?**, and a profound lack of authorial agency. Furthermore, existing interfaces treat news consumption as an isolated, ephemeral task rather than a collaborative, shareable experience **?**.

To address these gaps, this project introduces **Kişisel**, a desktop-first, widget-based social curation platform. Grounded in the HCI principles of user control, adaptive interfaces, and designed serendipity, Kişisel empowers readers to become active commentators, or "gatewatchers" **?**. By leveraging AI-generated summaries to manage cognitive load and introducing a dedicated Editorial Widget ecosystem, users can curate news streams, draft personal commentary, and publish their customized layouts as shareable digital newspapers. This document details the problem space, theoretical background, system architecture, and rigorous evaluation strategy for a platform that redefines digital reading from a passive feed into a user-owned information architecture.

1 Problem Statement

The contemporary news consumer inhabits a paradox: while access to information has never been greater, the capacity for meaningful engagement has never been more constrained. Mainstream platforms such as Google News, Apple News, and Flipboard impose algorithmic curation upon passive users, systematically removing agency over how, what, and how deeply one reads. Simultaneously, static, one-size-fits-all layouts fail to accommodate the cognitive diversity of readers—a casual browser and a deep researcher are forced into identical interface structures, despite fundamentally divergent information needs.

Four overlapping problems define this design space:

Information Overload and Cognitive Fatigue. The sheer volume of daily news exceeds the cognitive capacity of any individual reader. Without tools to modulate the depth and density of content, users experience what scholars term *news fatigue*—a withdrawal from news consumption driven not by disinterest, but by interface-induced exhaustion **?**. The paradox is that abundance, when unmediated, becomes its own barrier.

Passive Consumption and the Filter Bubble Effect. When layout and content selection are entirely controlled by opaque platform algorithms, users become progressively confined to content that mirrors and reinforces existing interests and beliefs. Pariser **?** coined *filter bubble* to describe this phenomenon; subsequent empirical work confirms that such systems amplify negative news cycles and reduce exposure to ideologically diverse content. Critically, the problem is not personalization itself—it is non-negotiable personalization that removes user agency.

Lack of User Voice and Authorial Agency. Existing news platforms position users as consumers, not contributors. A reader who wishes to contextualize an article, offer critical commentary, or annotate a story for their community has no native mechanism to do so within the interface. This one-directional consumption model represents a missed opportunity for deeper engagement. The ability to add personal commentary transforms reading from a passive act into an active, meaning-making practice—yet this capacity remains conspicuously absent from mainstream news aggregation.

Lack of Shareable, User-Owned Information Architectures. Existing platforms treat news layouts as ephemeral UI states, not as persistent, shareable artifacts. No dominant platform allows a user to construct a curated news experience as a document—something that can be shared, remixed, or subscribed to by others. The social curation literature identifies this as a significant gap: collaborative and social dimensions of news curation remain underexplored in desktop-first interface design **??**.

This project proposes Kişisel as a widget-based, desktop-first social curation platform that addresses these four problems through: (1) full user control over layout composition, (2) structural serendipity mechanisms to counter filter bubbles, (3) AI-generated summaries with source attribution as the primary reading unit, (4) adaptive reading modes that match interface density to user intent, (5) user-authored editorial commentary as a first-class content type, and (6) shareable personal newspapers as persistent social artifacts that position users as curators rather than consumers.

2 Literature Review

2.1 Widget-Based Dashboard Customization

The widget-based dashboard paradigm has been extensively studied in enterprise and productivity contexts. Research by Sloep et al. **?** on the *AWESOME* awareness dashboard established that widget-based personal environments empower users to self-organize information according to individual cognitive priorities—a finding that directly motivates the modular layout design of this platform. Effective customization requires: organization of data by user-defined importance, freedom to change representational form, and a widget bank with consistent sizing constraints—principles that map directly to the proposed drag-and-drop widget system.

UX best-practice literature consistently recommends drag-and-drop interfaces for widget placement, support for saving multiple custom views, and avoidance of information overload through controlled widget density. Crucially, Norman's **?** principle of *user control and freedom* underpins the entire customization paradigm: users must feel that the interface is *their* space, not a space they are merely permitted to occupy.

Design Implication: The platform's grid system should offer meaningful size flexibility (compact / standard / wide widget modes) to accommodate both information-dense power users and lighter casual readers within the same layout framework.

2.2 Filter Bubbles and Serendipitous Discovery

Pariser's **?** filter bubble thesis remains the most cited theoretical anchor in personalized news research. Empirical follow-up studies have complicated the picture—Möller et al. (2018) found that *self-selected* personalization (where users choose their own topics) produces less ideological isolation than algorithmic personalization—but the risk of echo chambers is particularly pronounced when users are *unaware* that filtering is occurring.

The proposed solution—a mandatory *Popular News* and *Random News* section within the customizable layout—is grounded in the HCI concept of **serendipitous information encountering ?**. Serendipity in information systems is not accidental; it must be *designed*. McCay-Peet and Toms define it as a system property that facilitates "unexpected, positive discoveries" by deliberately exposing users to content outside their established interest graph. The Popular + Random section resolves the tension between *perceived relevance* and *peripheral awareness* by making the serendipity mechanism transparent and user-visible rather than hidden in the algorithm.

Design Implication: The Popular and Random widgets should be non-removable or gently nudged as default components in new layouts, framing serendipity as a feature rather than an opt-in afterthought.

2.3 AI Summarization as Primary Reading Unit

The use of AI-generated article summaries as the *primary interface unit*—with click-through to the original source—positions this platform at the intersection of cognitive load theory and human-AI interaction design.

Miller's (1956) law of cognitive load and Sweller's (1988) Cognitive Load Theory both establish that human working memory is limited. A feed of full articles exceeds this capacity; a feed of well-structured summaries that allow self-paced depth control does not. This "progressive disclosure" model—where

users skim a summary and choose whether to engage with the full source—is a well-established information architecture pattern **?**.

The AI summarization layer introduces a critical trust dimension. Darejeh et al. **?** note that while AI tools can dramatically reduce content processing time, interfaces that fail to make AI involvement *visible* risk eroding user trust when errors occur. The mitigation is built into the design contract itself: **summaries are clearly labeled as AI-generated previews, and the canonical source is always one click away.** This preserves source authority while reducing the friction of initial engagement.

Furthermore, the summary-first model directly addresses the *news fatigue* problem. Newman et al.'s Reuters Institute Digital News Report **?** found that the primary driver of news avoidance is not distrust of media, but the *emotionally draining* and *time-consuming* nature of news consumption.

Design Implication: Each summary card should visually distinguish the AI-generated summary layer from source metadata (publication name, date, author), reinforcing that the summary is a navigational aid, not the authoritative text.

2.4 Adaptive Reading Modes

The concept of adaptive interfaces—systems that adjust their presentation in response to user context or stated intent—has a substantial HCI literature rooted in Benyon and Murray's (1993) foundational work on user modeling. In the context of news reading, the key contextual dimension is *reading intent*: a user scanning headlines during a commute has fundamentally different cognitive needs than a user doing background research.

Shneiderman's **?** Visual Information Seeking mantra—*overview first, zoom and filter, then details on demand*—maps precisely onto the three reading modes proposed for this platform: headline/summary mode (overview), standard card mode (zoom), and full-context mode with related articles (details on demand). This three-tier model has been validated in news interface contexts by Diakopoulos et al. (2012), who found that users significantly prefer interfaces that allow them to modulate reading depth rather than committing to a single level of engagement.

Design Implication: Mode switching should be a persistent, low-friction UI control—ideally a keyboard shortcut as well as a visible toggle—and the selected mode should persist across sessions as a user preference, not reset on each visit.

2.5 Shareable Personal Newspapers as Social Artifacts

The most novel contribution of this platform is the framing of a personal news layout as a *shareable social artifact*, positioning the project at the intersection of social computing and personal information management research.

Schneider et al. **?** studied the *Acropolis* platform—a social computing environment for collaborative news curation—and found that the ability to build and share narrative structures around news significantly increased civic engagement and reading depth. Their design recommendations include: clear attribution of curation decisions, easy remix mechanics, and asymmetric collaboration support (viewers can subscribe without needing to curate themselves).

The broader social curation literature **?** establishes that *user-distributed content*—the act of selecting and reframing content for a social audience—is a distinct and meaningful form of news participation. By allowing users to publish their widget layout as a named, subscribable newspaper, the platform transforms its users from consumers into curators, which Bruns **?** terms *gatewatchers* in his theory of networked journalism.

Design Implication: Shared newspapers should have a persistent public URL, a visual "cover" that reflects the layout and topic mix, and a one-click subscribe mechanism that lets visitors adopt the layout or follow the curated content feed.

2.6 The Curator as Commentator

Bruns **?** introduced the concept of *gatewatchers* in networked journalism—users who actively select, reframe, and contextualize content for their social audiences, distinct from passive consumers or algorithmic sharers. Villi **?** further establishes that user-distributed content represents a distinct and meaningful form of news participation. By allowing users to attach their own editorial commentary to articles within their personal newspapers, Kişisel transforms users from consumers into curators and commentators, activating the social and authorial dimensions of news engagement that existing platforms suppress.

Design Implication: Editorial commentary must be integrated as a native widget type, enabling users to add rich-text annotations that appear alongside the articles they curate, with clear attribution of authorship.

3 Project Stages

The project follows a four-stage iterative design process, consistent with the double diamond model widely adopted in HCI research (Design Council, 2005). Each stage produces a concrete deliverable that informs the next.

Stage 1 — Discovery & Requirements (Weeks 1–2). The first stage establishes the user research foundation. Through competitive analysis of existing platforms (Google News, Feedly, Flipboard, Artifact) and a structured user survey ($n \geq 10$), we identify how target users currently consume news, what frustrations they encounter, and what customization behaviors they perform manually. Survey findings are synthesized into at least three user personas—crucially including the "Thought Leader/Commentator" persona who requires advanced tools for annotation and public sharing. Task analysis will cover primary flows: composing a layout, discovering serendipitous content, switching reading modes, authoring personal commentary, and sharing a newspaper.

Stage 2 — Design & Prototyping (Weeks 3–4). With requirements established, the design stage proceeds in two steps. Week 3 focuses on lo-fi wireframe sketches and system architecture decisions (data flow, component hierarchy, API contracts). Week 4 moves directly into code-based rapid prototyping: React scaffolding, a basic widget grid, routing, and initial state management. Design decisions are explicitly mapped to the literature: serendipity sections are anchored to McCay-Peet & Toms **?**; the reading mode hierarchy follows Shneiderman's **?** model; the shareable newspaper URL pattern draws on Schneider et al. **?**; and the prominent placement of editorial tools is grounded in Bruns' **?** gatewatching theory.

Stage 3 — Implementation (Weeks 5–7). Implementation spans three focused weeks. Week 5 covers backend and data layer: NestJS API setup, PostgreSQL schema, Redis caching, RSS feed ingestion pipeline, and AI summarization API integration. Week 6 focuses on the frontend: reading mode context provider, Editorial Widget, the Popular/Random section, and UI polishing with real RSS seed data. Week 7 is reserved for full-stack integration and QA: end-to-end flow testing, cross-component state verification, and resolution of integration bugs before the handoff to usability testing.
Key deliverables: deployed functional prototype (localhost or Vercel staging), component library, API integration, seeded database.

Stage 4 — Evaluation & Iteration (Weeks 8–10). The functional prototype is evaluated through a moderated usability study. Findings are categorized using Nielsen's (1994) severity ratings, and a final iteration round addresses critical and major usability issues before the final submission. Quantitative metrics (task completion rate, time-on-task, SUS score) and qualitative findings (think-aloud observations, post-test interviews) are synthesized into the final evaluation report.

To ensure scientific rigor, authorial agency and user engagement will be measured through objective behavioral metrics:

- **Widget Adoption Rate:** The percentage of users who voluntarily incorporate the editorial widget into their custom layout during free-exploration tasks.
- **Commentary Task Performance:** Task completion rates and time-on-task for successfully authoring and saving a brief (1–2 sentence) note.
- **Sharing Inclusion Rate:** The proportion of generated public "newspaper" links that include user-authored text alongside aggregated news links, providing quantifiable evidence of the transition from passive consumer to active curator.

Key deliverables: usability study report, severity-rated issue list, iterated prototype, final presentation.

4 System Architecture and Tools

4.1 Frontend

The platform is a desktop-first single-page application built with **React** and **TypeScript**. The widget grid system is implemented using **React Grid Layout**, which provides drag-and-drop placement, resize handles, and persistent layout serialization. Component state is managed with **Zustand** for lightweight global state (current reading mode, active layout, user preferences) and **TanStack Query** for server state and feed caching.

The adaptive reading mode system operates as a React context provider that wraps all widget components, injecting a `readingMode` value (`headline | summary | full`) which each widget consumes to adjust its render depth. Mode switching is therefore a single state change that propagates to all widgets simultaneously—no re-fetch required.

4.2 Backend & Data Layer

The backend is a lightweight **NestJS** API responsible for user authentication, layout persistence, newspaper sharing (generating public slugs), and RSS feed aggregation. News content is pulled from real RSS feeds across configurable source categories. Feed data is cached in **Redis** with a short TTL (15–30 min) to avoid hammering sources and to ensure Popular section ranking reflects genuine recency.

The Popular News section is ranked by a composite score: recency $\times$ estimated engagement (based on cross-source publication frequency of the same story). The Random News section samples uniformly from the full feed corpus, intentionally bypassing user preference data.

4.3 AI Summarization

Each feed item is passed through an **LLM summarization pipeline** (OpenAI GPT-4o-mini or a self-hosted Mistral variant) to generate a 2–3 sentence summary. Summaries are generated on ingest and cached alongside the article metadata, so the UI never blocks on AI generation latency. Each summary card visually distinguishes the AI layer (summary text) from source metadata (publication, author, date), preserving source authority as discussed in Section 2.3.

4.4 Newspaper Sharing

A user's widget layout—including source configuration, widget positions, sizes, and reading mode preference—is serialized as a JSON document and stored with a unique public slug (`/newspaper/:slug`). Visitors to a shared newspaper see a read-only version of the layout with live-updated content. A one-click "Use this layout" button lets visitors fork the layout into their own account.

4.5 Technology Stack Summary

Table 1: Technology Stack

Layer	Technology
Frontend framework	React + TypeScript
Widget grid	React Grid Layout
State management	Zustand + TanStack Query
Backend	NestJS
Database	PostgreSQL (layouts, users)
Cache	Redis (feed data)
AI summarization	OpenAI API (GPT-4o-mini)
Feed ingestion	RSS parser + cron jobs
Deployment	Vercel (frontend) + Railway / Render (backend)

5 Experiments and Test Results

5.1 Evaluation Strategy

The evaluation follows a **mixed-methods usability study** design, combining quantitative performance metrics with qualitative observational data. This approach is consistent with Lazar et al. (2017) recommendations for HCI evaluation studies, which argue that task metrics alone are insufficient to capture the full user experience—particularly for exploratory, open-ended interfaces like a customizable dashboard.

All testing is conducted with the functional Stage 3 prototype on desktop (minimum 1280 px viewport). Participants are recruited from the university student population ($n = 8$–10), representing the target demographic of news-reading desktop users. Sessions are conducted remotely via screen share or in-person, with screen recording and audio consent obtained.

5.2 Usability Testing Protocol

Each session consists of four parts:

Part 1 — Pre-test questionnaire (5 min). Participants complete a short questionnaire on their current news consumption habits, platform preferences, and self-reported comfort with customizable interfaces, providing baseline data for interpreting task performance.

Part 2 — Scenario-based tasks (25 min). Participants are given six structured scenarios derived from the Stage 1 task analysis (Table 2).

Table 2: Usability Test Tasks

Task	Scenario Description	Primary Metric
T1 — Layout composition	Build a layout that reflects your ideal morning news read	Time-on-task, error count
T2 — Reading mode switch	Switch all widgets to headline-only mode	Completion rate, discoverability
T3 — Serendipity engagement	Find a news item outside your usual interests using the platform	Success rate, widget used
T4 — Source navigation	Read the full article for a story that interests you	Click path, time to source
T5 — Newspaper sharing	Share your layout with a friend and show them how to subscribe	Completion rate, errors
T6 — Editorial creation	Add an Editorial widget, write a brief comment, and save it	Completion rate, errors

The **think-aloud protocol** is used throughout, with the facilitator prompting participants to verbalize their reasoning without providing guidance.

Part 3 — System Usability Scale (5 min). Upon task completion, participants complete the 10-item SUS questionnaire **?**. The SUS provides a standardized, single-number usability score (0–100) enabling cross-study comparison. A target score of ≥ 68 (the industry average) is set as the minimum acceptable threshold; a score of ≥ 80 is the project's stretch target.

Part 4 — Semi-structured post-test interview (5 min). Participants are asked open-ended questions targeting the three core design decisions: widget customization freedom, the serendipity sections, and the reading mode system. Interview responses are transcribed and analyzed using affinity diagramming to surface recurring themes.

5.3 Metrics Summary

Table 3: Evaluation Metrics and Targets

Metric	Measurement Method	Target
Task completion rate	Observed pass/fail per task	$\geq 80\%$ across all tasks
Time-on-task (T1)	Stopwatch from task prompt to declared completion	< 3 min
SUS score	Standard 10-item questionnaire	≥ 68 (stretch: ≥ 80)
Error rate	Count of incorrect actions per task	≤ 2 errors avg. on T2, T5
Serendipity engagement rate	% of participants who click a Popular or Random item	$\geq 40\%$
Discoverability of mode switcher	% who locate the control without prompting	$\geq 70\%$

5.4 Analysis Plan

Quantitative data (task completion, time, SUS) will be reported descriptively (mean, standard deviation) given the small sample size—inferential statistics are not appropriate at $n < 15$. Usability issues identified in the think-aloud and interview data will be categorized using **Nielsen's (1994) severity ratings** ($0 =$ not a usability problem $\rightarrow 4 =$ usability catastrophe). Issues rated 3 or 4 will be addressed in the Stage 4 iteration round before final submission.

6 Weekly Schedule

Table 4: 10-Week Project Schedule

Week	Phase	Goals	Deliverables
1	Discovery	Competitive analysis of 4 platforms (Google News, Feedly, Flipboard, Artifact); design and distribute user survey ($n \geq 10$); initial research gap documentation	Competitive analysis report, survey instrument
2	Requirements	Analyze survey results; synthesize 3 user personas (including Thought Leader/Commentator); map primary task flows; finalize feature scope	Personas, task flow diagrams, finalized feature list
3	Lo-fi Design & Architecture	Sketch lo-fi wireframes for all core flows; define system architecture, component hierarchy, API contracts, and data models	Lo-fi wireframe set, system architecture diagram, API contract document
4	Rapid Prototyping	Code-based rapid prototyping: React scaffolding, basic widget grid, routing, global state management skeleton, and Editorial Widget mockup	Interactive coded prototype covering all 6 primary task flows
5	Implementation — Backend	NestJS API setup; PostgreSQL schema; Redis caching layer; RSS feed ingestion pipeline; newspaper sharing slug generation; AI summarization API integration	Functional backend, API integrations, seeded database, working summarization pipeline
6	Implementation — Frontend	Reading mode context provider; drag-and-drop widget grid; Editorial Widget; Popular and Random sections; UI polishing with live RSS seed data	Fully functional frontend connected to backend; all 6 task flows operational
7	Integration & QA	Full-stack integration testing; end-to-end flow verification; cross-component state audits; resolution of integration bugs; staging deployment	Stable integrated prototype deployed on Vercel + Railway staging, QA report
8	Usability Testing	Recruit 8–10 participants; conduct moderated think-aloud usability sessions (T1–T6); collect SUS questionnaires and session recordings	Raw session recordings, SUS score sheets, interview transcripts
9	Analysis & Iteration	Transcribe and affinity-diagram interview data; severity-rate all issues per Nielsen (1994); fix all issues rated ≥ 3; re-test critical fixes	Severity-rated issue log, iterated prototype, regression test results
10	Final Report & Presentation	Synthesize quantitative and	Final evaluation report, pre-

Gantt Chart

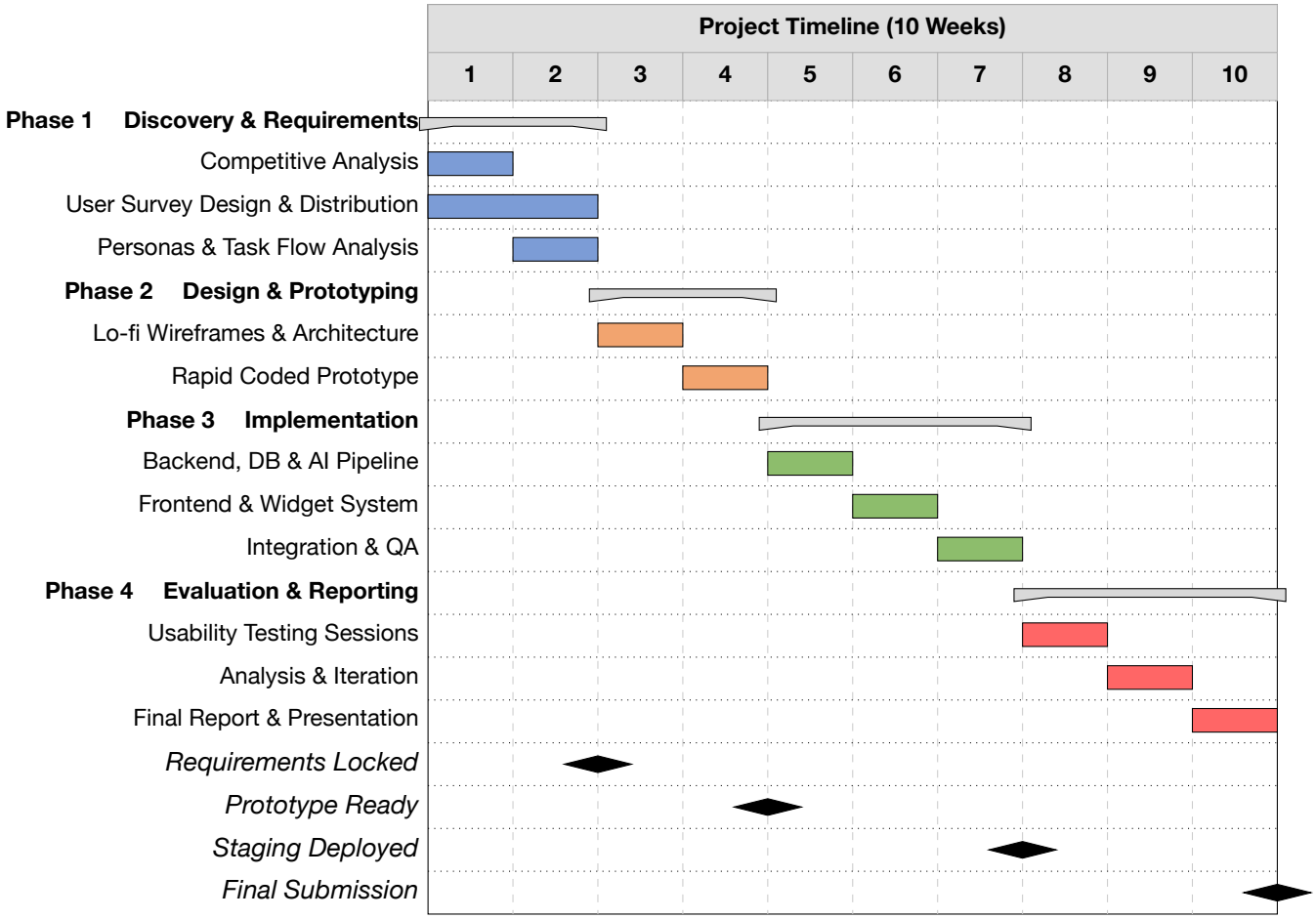


Figure 1: Gantt chart showing the four project phases across ten weeks. Milestones mark key hand-off points between phases.